

Laboratory Access Control System - Project Report

1. Structure of Project

The project is structured into two main modules: **Client** (Edge Device) and **Server** (Central Authentication Server), ensuring low latency and offloading heavy computation.

Directory Tree Overview

- **client/**: Lightweight client application running on edge devices (e.g., Raspberry Pi or local PC with camera).
 - **core/**: Core functionalities for the edge device.
 - **camera.py**: Video capture and frame management.
 - **detector.py**: MediaPipe-based face detection and landmark extraction.
 - **embedder.py**: FaceNet embedding generation via ONNX and frame aggregation.
 - **display.py**: UI and overlay management for the video feed.
 - **ws_client.py**: WebSocket communication with the central server.
 - **utils/**: Utilities for image processing.
 - **alignment.py**: Face alignment using affine transformations.
 - **main.py**: Entry point for the client process.
 - **enroll.py**: Script to enroll new users via the client camera.
 - **test_connection.py, check_model.py, inspect_model.py, collect_training_data.py**: Various helper and diagnostic scripts.
 - **config.py**: Configuration loading from environment variables.
 - **server/**: Centralized authentication and database management server.
 - **core/**: Authentication and matching algorithms.
 - **jwt.py**: JWT token generation and validation.
 - **matcher.py**: Cosine similarity logic and access decision rules.
 - **security.py**: Password hashing and verification.
 - **db/**: Database interactions.
 - **models.py**: SQLAlchemy ORM models (e.g., Users, Embeddings, Logs).
 - **session.py**: Asynchronous database connection management.
 - **routers/**: FastAPI route handlers.
 - **enroll.py**: API endpoints for user enrollment.
 - **verify.py**: WebSocket and REST endpoints for face verification.
 - **mfa.py**: Multi-Factor Authentication handling.
 - **schemas/**: Pydantic models for request/response validation.
 - **main.py**: FastAPI application setup and lifecycle events.
 - **config.py**: Environment configuration variables.
 - **alembic/, alembic.ini**: Database migration scripts.
 - **Root Files**:
 - **docker-compose.yml**: Container orchestration setup.
 - **.env.example**: Template for environment variables.
 - **README.md**: Project documentation.
 - **models/**: Directory holding compiled ONNX machine learning models.
-

2. Programming Language, Libraries & Packages

Programming Language

- **Python 3.10+**: Used universally across both client and server codebases, heavily leveraging modern features such as Type Hinting, Dataclasses, and Asynchronous I/O (`asyncio`).

Client Dependencies (Edge Processing)

- **OpenCV (opencv-python-headless)**: Image processing, bounding box rendering, affine transformations, and image resizing.

- **MediaPipe (mediapipe)**: Fast and robust on-device face detection and facial landmark extraction.
- **ONNX Runtime (onnxruntime)**: Execution of the FaceNet machine learning model for generating embeddings.
- **WebSockets (websockets)**: Real-time, low-latency communication with the server.
- **NumPy (numpy)**: Matrix operations, frame aggregation, and data normalization.
- **Pillow (Pillow)**: Additional image manipulation where required.

Server Dependencies (Backend API & Authentication)

- **FastAPI (fastapi) & Uvicorn (uvicorn[standard])**: High-performance asynchronous web framework and ASGI server for handling REST and WebSocket connections.
- **SQLAlchemy (sqlalchemy[asyncio])**: Asynchronous Object-Relational Mapping (ORM) for PostgreSQL database interactions.
- **Alembic (alembic)**: Database schema migration tool.
- **Asyncpg (asyncpg)**: High-performance PostgreSQL database driver for Python.
- **Pydantic (pydantic) & Pydantic-Settings (pydantic-settings)**: Data validation, serialization, and settings management.
- **Passlib (passlib[bcrypt])**: Secure password hashing for user management (MFA/login).
- **PyJWT (pyjwt)**: Generation and decoding of JSON Web Tokens for API security.
- **NumPy (numpy)**: Mathematical operations, specifically for cosine similarity calculations.

3. Algorithms & Functions

Client-Side Algorithms

1. Face Detection & Landmark Extraction

- **File**: `client/core/detector.py`
- **Logic**: Uses MediaPipe's `FaceDetection` and `FaceMesh` to identify the largest face in a video frame. It pads the bounding box by 20% to capture the full head and extracts coordinates for the left eye (index 33) and right eye (index 263).

2. Face Alignment (Affine Transformation)

- **File**: `client/utils/alignment.py`
- **Logic**: Computes the angle between the two eyes. Using OpenCV's `getRotationMatrix2D` and `warpAffine`, it rotates the image so that the eyes are perfectly horizontal. This minimizes false rejections caused by a user tilting their head.

3. FaceNet Embedding Generation

- **File**: `client/core/embedder.py`
- **Logic**: Takes an aligned face crop, resizes it to 112x112, and normalizes pixel values to `[-1, 1]`. It then passes the tensor through an ONNX model, which outputs a unique 512-dimensional vector representation of the face.

4. Frame Aggregation (Noise Cancellation)

- **File**: `client/core/embedder.py` (`FrameAggregator`)
- **Logic**: Buffers `N` (default 7) sequential facial embeddings over time. It calculates the mean across all vectors and applies L2-normalization to produce a single, highly stable unit vector, preventing authentication failures from momentary blurs or bad lighting.

Server-Side Algorithms

1. Cosine Similarity Matching

- **File**: `server/core/matcher.py` (`cosine_similarity`)
- **Logic**: Calculates the cosine of the angle between the incoming 512-dimensional embedding and the stored embeddings in the database using the dot product formula: $(A \cdot B) / (||A|| * ||B||)$. It outputs a score between -1.0 and 1.0, where 1.0 represents an exact match.

2. Three-Zone Decision Logic

- **File**: `server/core/matcher.py` (`decide`)
- **Logic**: Uses predefined thresholds to categorize the similarity score into three strict zones:
 - **Zone A (ALLOW)**: Score is greater than `THRESHOLD_ALLOW`. The user is instantly authenticated.
 - **Zone B (MFA_CHALLENGE)**: Score is between `THRESHOLD_MFA` and `THRESHOLD_ALLOW`. The system is unsure and prompts the user for a secondary factor (e.g., PIN).
 - **Zone C (DENY)**: Score is below `THRESHOLD_MFA`. The face does not match any enrolled user, and access is immediately denied.